

AWS SageMaker Deployment

01-ml-infrastructure/sagemaker/deployment

Overview

SageMaker Deployment turns trained models into managed inference endpoints or batch inference jobs. It packages model artifacts, container logic, scaling behavior, and network settings into a serving configuration.

The value is operational consistency. Teams can promote models through controlled environments instead of treating deployment as a manual afterthought.

Core Concepts

- A deployment usually starts with a model definition that points to the serving image and the model artifact in S3.
- Endpoint configurations describe instance types, variant weights, autoscaling behavior, or specialized modes such as asynchronous or serverless inference.
- SageMaker supports patterns such as multi model endpoints, blue green style updates through variants, and data capture for later monitoring.

How It Works

- After a model is approved, the deployment system creates or updates the endpoint configuration and then rolls traffic to the new endpoint variant.
- Clients send inference requests to the hosted endpoint, while logs, metrics, and optionally request payload capture flow to surrounding AWS services.
- For offline scoring, the same model package can be used in batch transform rather than a permanently running real time endpoint.

AI and ML Relevance

- Deployment design affects latency, throughput, cost, and release safety. The right mode depends on traffic profile, payload size, and how quickly results are needed.
- ML teams also care about safe promotion, shadowing, rollback, and post deployment quality checks because correctness drift can be more subtle than simple uptime failures.

Operational Notes

- Use autoscaling carefully. If cold starts or model load time are high, a low minimum instance count can cause unstable latency.
- Capture inference data where governance allows it so you can debug failure cases, audit outcomes, and build monitoring or retraining signals.
- Separate release approval from endpoint mutation. Production deployment should be traceable to a specific model version and image version.

What To Remember

- Training creates a model artifact; deployment turns it into a callable service contract.
- Real time, asynchronous, serverless, and batch inference exist for different traffic and cost profiles.
- The hardest part is rarely the API call. It is the release discipline around versions, rollbacks, and observability.